

S140-C R1 - Activation-envelope repair replay

Wolfram source and recorded execution result

Retrospective replay of the archived S140-B high-prefix failure. The original failure is preserved separately.

Wolfram runner source

```
ClearAll["Global`*"];

rootDirectory = DirectoryName[DirectoryName[$InputFileName]];
protocolPath = FileNameJoin[{rootDirectory, "protocol", "frozen_protocol.json"}];
publicPath = FileNameJoin[{rootDirectory, "input", "public_tasks.json"}];
resultPath = FileNameJoin[{rootDirectory, "results", "kernel_native_concept_result.json"}];
conceptLibraryPath = FileNameJoin[{rootDirectory, "library", "kernel_induced_concepts.json"}];
oracleResponderPath = FileNameJoin[{rootDirectory, "source", "oracle_responder.py"}];
oracleRequestPath = FileNameJoin[{rootDirectory, "oracle", "runtime_request.json"}];
oracleResponsePath = FileNameJoin[{rootDirectory, "oracle", "runtime_response.json"}];
oracleLogPath = FileNameJoin[{rootDirectory, "oracle", "query_log.jsonl"}];
protocol = Import[protocolPath, "RawJSON"];
public = Import[publicPath, "RawJSON"];
protocolHash = IntegerString[FileHash[protocolPath, "SHA256"], 16, 64];
runnerHash = IntegerString[FileHash[$InputFileName, "SHA256"], 16, 64];
oracleHash = IntegerString[FileHash[oracleResponderPath, "SHA256"], 16, 64];
If[public["ProtocolSHA256"] != protocolHash, Print["FATAL: public/protocol mismatch"]; Exit[2]];
If[runnerHash != protocol["FrozenWolframRunnerSHA256"], Print["FATAL: runner differs from frozen protocol"]; Exit[3]];
If[oracleHash != protocol["FrozenOracleResponderSHA256"], Print["FATAL: oracle differs from frozen protocol"]; Exit[4]];

ContextColorsS139[level_Integer] := {6, 7, 9}[[Mod[level, 3] + 1]];

GridDigestG3[grid_List] := IntegerString[Hash[StringRiffle[ToString /@ Flatten[grid], ",", "SHA256"], 16, 64];
AllPrefixesG3[counts_List, 0] := {};
AllPrefixesG3[counts_List, length_Integer] := Tuples[Table[Range[0, counts[[index]] - 1], {index, length}]];

MakeInputG3[task_Association, spec_Association] := Module[
  {h = task["GridHeight"], w = task["GridWidth"], grid, shape, top, left, kind, prefix},
  grid = ConstantArray[0, {h, w}];
  Do[Do[grid[[h - level, 3 + 4 level + value]] = ContextColorsS139[level],
    {value, 0, task["ContextCounts"][[level + 1]] - 1}], {level, 0, Length[task["ContextCounts"]] - 1}];
  kind = spec["Kind"];
  shape = Which[kind == "TEST", task["TestShape"], kind == "TRAIN", task["TrainShape"], True, task["ProbeShape"]];
  top = task["TargetTop"]; left = task["TargetLeft"];
  Do[grid[[top + cell[[1]] + 1, left + cell[[2]] + 1]] = task["TargetColor"], {cell, shape}];
  prefix = Lookup[spec, "Prefix", {}];
  Do[grid[[2 + level, 3 + 4 level + prefix[[level + 1]]]] = 8, {level, 0, Length[prefix] - 1}];
  Which[
    kind == "CALIBRATE", grid[[1, spec["Level"] + 1]] = 9,
    kind == "DECISION", grid[[1, w]] = 8,
    kind == "GLOBAL_DECISION", grid[[1, w - 1]] = 4,
    kind == "GLOBAL_BIT0", grid[[1, w - 1]] = 8,
    kind == "GLOBAL_BIT1", grid[[1, w - 2]] = 8,
    kind == "NUISANCE", grid[[1, w]] = 7,
    kind == "TEST", grid[[1, w]] = 5,
    kind == "TRAIN", Null,
    True, Print["FATAL: unknown spec kind"]; Exit[10]
  ]; grid
];

ApplyProgramG3[task_Association, program_Association, spec_Association] := Module[
  {grid, output, kind, prefix, level, value, transform, positions, r0, r1, c0, c1, mapped},
  grid = MakeInputG3[task, spec]; output = grid; kind = spec["Kind"];
  prefix = Lookup[spec, "Prefix", {}];
  If[kind == "CALIBRATE", level = spec["Level"];
    If[!MemberQ[Lookup[task, "CorruptedCalibrationLevels", {}], level] &&
      Take[program["Keys"], level] == prefix, value = program["Keys"][[level + 1]];
      output[[6 + level, 3 + 4 level + value]] = ContextColorsS139[level]; Return[output]];
  If[kind == "NUISANCE", output[[9, 13 + program["Nuisance"]]] = 9; Return[output]];
  If[MemberQ[{"GLOBAL_BIT0", "GLOBAL_BIT1"}, kind],
    If[program["Decision"] == If[kind == "GLOBAL_BIT0", 0, 1],
      output[[10, Length[First[output]] - 1]] = 9; Return[output]];
  transform = MemberQ[{"TRAIN", "TEST", "GLOBAL_DECISION"}, kind] ||
    (kind == "DECISION" && Take[program["Keys"], Length[prefix]] == prefix);
  If[!transform, Return[output]];
  positions = Position[grid, task["TargetColor"], {2}];
  r0 = Min[positions[[All, 1]]]; r1 = Max[positions[[All, 1]]];
  c0 = Min[positions[[All, 2]]]; c1 = Max[positions[[All, 2]]];
```

```

mapped = Switch[program["Decision"], 0, positions,
  1, {#[[1]], c0 + c1 - #[[2]]} & /@ positions,
  2, {r0 + r1 - #[[1]], #[[2]]} & /@ positions];
Do[output[[position[[1]], position[[2]]]] = 0, {position, positions}];
Do[output[[position[[1]], position[[2]]]] = task["TargetColor"], {position, mapped}]; output
];

BuildQueriesG3[task_Association] := Module[{depth, disabled, specs, trainHashes, rows},
  depth = Length[task["ContextCounts"]];
  disabled = Lookup[task, "DisabledCalibrationLevels", {}];
  specs = Flatten[Table[<|"Kind" -> "CALIBRATE", "Level" -> level, "Prefix" -> prefix|>,
    {level, Select[Range[0, depth - 1], !MemberQ[disabled, #] &]},
    {prefix, AllPrefixesG3[task["ContextCounts"], level]]}, 1];
  specs = Join[specs, (<|"Kind" -> "DECISION", "Level" -> Null, "Prefix" -> #|> & /@
    AllPrefixesG3[task["ContextCounts"], depth]),
    If[TrueQ[Lookup[task, "GlobalDecisionProbe", False]],
      <|"Kind" -> "GLOBAL_DECISION", "Level" -> Null, "Prefix" -> {}|>, {}],
    If[TrueQ[Lookup[task, "GlobalBinaryDecisionProbes", False]],
      <|"Kind" -> "GLOBAL_BIT0", "Level" -> Null, "Prefix" -> {}|>,
      <|"Kind" -> "GLOBAL_BIT1", "Level" -> Null, "Prefix" -> {}|>, {}],
    <|"Kind" -> "NUISANCE", "Level" -> Null, "Prefix" -> {}|>];
  trainHashes = GridDigestG3 /@ Lookup[task["InitialTrain"], "Input"];
  rows = Map[Function[spec, With[{grid = MakeInputG3[task, spec]}, <|
    "Kind" -> spec["Kind"], "Level" -> spec["Level"], "Prefix" -> spec["Prefix"],
    "Input" -> grid, "InputSHA256" -> GridDigestG3[grid]|>]], specs];
  SortBy[Select[rows, !MemberQ[trainHashes, #1["InputSHA256"]] &], #1["InputSHA256"] &];
];

TrainingExactG3[task_Association, program_Association] := And @@ Map[
  ApplyProgramG3[task, program, #1["Spec"]] === #1["Output"] &, task["InitialTrain"]];

BuildModelsG3[task_Association, queries_List] := Module[{keyRows, programs, exact},
  keyRows = AllPrefixesG3[task["ContextCounts"], Length[task["ContextCounts"]]];
  programs = Flatten[Table[<|"Keys" -> keys, "Decision" -> decision, "Nuisance" -> nuisance|>,
    {keys, keyRows}, {decision, 0, 2}, {nuisance, 0, task["NuisanceCount"] - 1}], 2];
  exact = Select[programs, TrainingExactG3[task, #1] &];
  Map[Function[program, With[{testSpec = <|"Kind" -> "TEST", "Level" -> Null, "Prefix" -> {}|>},
    With[{prediction = ApplyProgramG3[task, program, testSpec]}, <|
      "ModelKey" -> StringRiffle[ToString /@ Join[program["Keys"], {program["Decision"], program["Nuisance"]}]], ":"],
      "Program" -> program, "DecisionLabel" -> GridDigestG3[prediction], "TestPrediction" -> prediction,
      "QueryPredictions" -> Association@Map[Function[query, query["InputSHA256"] ->
        ApplyProgramG3[task, program, query]], queries]|>]], exact];
];

DecisionCountG3[models_List] := Length[DeleteDuplicates[Lookup[models, "DecisionLabel"]]];
PredictionG3[model_Association, queryHash_String] := model["QueryPredictions"][queryHash];
BranchesG3[models_List, queryHash_String] := GatherBy[models, PredictionG3[#, queryHash] &];
UnsolvedG3[] := <|"Solvable" -> False, "RequiredDepth" -> Null, "FirstInputSHA256" -> Null|>;
$PlanMemoG3 = <||>;
$PlanCountersG4 = <||>;
$QueryByHashG4 = <||>;
$UseConceptsG4 = False;
conceptLibrary = <|"Concepts" -> {}|>;

ResetPlanCountersG4[] := ($PlanCountersG4 = <|
  "ExpandedStateCount" -> 0, "QueryEvaluationCount" -> 0,
  "OutcomeBranchEvaluationCount" -> 0, "ConceptApplicableStateCount" -> 0,
  "ConceptInstantiatedQueryCount" -> 0, "ConceptNoCandidateStateCount" -> 0,
  "ConceptPreferredQueryRejectedCount" -> 0,
  "ConceptActivationEnvelopeRejectedStateCount" -> 0,
  "ConceptActivationEnvelopeRejectedQueryCount" -> 0|>);
IncrementCounterG4[key_String, amount_Integer : 1] :=
  ($PlanCountersG4[key] = $PlanCountersG4[key] + amount);

CommonPrefixG4[models_List] := Module[{keys, prefix = {}, values},
  keys = Lookup[Lookup[models, "Program"], "Keys"];
  If[Length[keys] === 0, Return[prefix]];
  Do[values = DeleteDuplicates[keys][[All, index]]];
  If[Length[values] != 1, Break[]]; AppendTo[prefix, First[values]];
  {index, 1, Length[First[keys]]}; prefix
];

RelationalFeaturesS140[models_List, queryHash_String, planningDepth_Integer] := Module[

```

```

{known, coordinateCount, branchRows, branchKnown, branchDecisions, currentDecisions,
 worstKnown, worstDecisions},
known = Length[CommonPrefixG4[models]];
coordinateCount = Length[First[models]["Program"]["Keys"]];
branchRows = BranchesG3[models, queryHash];
branchKnown = Length[CommonPrefixG4[#]] & /@ branchRows;
branchDecisions = DecisionCountG3 /@ branchRows;
currentDecisions = DecisionCountG3[models];
worstKnown = If[Length[branchKnown] > 0, Min[branchKnown], known];
worstDecisions = If[Length[branchDecisions] > 0, Max[branchDecisions], currentDecisions];
<|"KnownPrefixLength" -> known, "CoordinateCount" -> coordinateCount,
 "RemainingCoordinateCount" -> coordinateCount - known,
 "PlanningBudget" -> planningDepth, "CurrentDecisionClassCount" -> currentDecisions,
 "WorstCaseKnownPrefixGain" -> worstKnown - known,
 "WorstCaseDecisionClassReduction" -> currentDecisions - worstDecisions,
 "WorstCaseRemainingDecisionClasses" -> worstDecisions,
 "OutcomeBranchCount" -> Length[branchRows]]>
];

AtomMatchesS140[atom_List, features_Association] := Module[{value = features[atom[[1]]],
 Switch[atom[[2]], "Equal", value == atom[[3]], "GreaterEqual", value >= atom[[3]],
 "LessEqual", value <= atom[[3]], _, False]
];

RuleMatchesS140[rule_Association, features_Association] := And @@
 (TrueQ[AtomMatchesS140[#, features]] & /@ rule["Conditions"]);

InventAtomsS140[featureRows_List] := Module[
 {atoms = {}, stateFeatures, queryFeatures, operators},
 stateFeatures = {"KnownPrefixLength", "CoordinateCount", "RemainingCoordinateCount",
 "PlanningBudget", "CurrentDecisionClassCount"};
 queryFeatures = {"WorstCaseKnownPrefixGain", "WorstCaseDecisionClassReduction",
 "WorstCaseRemainingDecisionClasses", "OutcomeBranchCount"};
 operators = {"Equal", "GreaterEqual", "LessEqual"};
 Do[AppendTo[atoms, {feature, operator, threshold}],
 {feature, Join[stateFeatures, queryFeatures]},
 {threshold, Sort[DeleteDuplicates[Lookup[featureRows, feature]]], {operator, operators}}];
 atoms
];

CandidateRulesS140[atoms_List] := Module[{rows, queryFeatures, queryAtoms},
 queryFeatures = {"WorstCaseKnownPrefixGain", "WorstCaseDecisionClassReduction",
 "WorstCaseRemainingDecisionClasses", "OutcomeBranchCount"};
 queryAtoms = Select[atoms, MemberQ[queryFeatures, #[[1]]] &];
 rows = (<|"Conditions" -> {#}> & /@ queryAtoms);
 Do[If[pair[[1, 1]] != pair[[2, 1]] &&
 (MemberQ[queryFeatures, pair[[1, 1]]] || MemberQ[queryFeatures, pair[[2, 1]]]),
 AppendTo[rows, <|"Conditions" -> Sort[pair|>]], {pair, Subsets[atoms, {2}]]];
 DeleteDuplicatesBy[rows, ToString[#1["Conditions"], InputForm] &]
];

InduceConceptsS139[events_List] := Module[
 {positives = {}, negatives = {}, candidates = {}, covered, taskSupport, falseCount,
 valid, uncovered, selected = {}, ranked, chosen, body, conceptID, eventIndex = 0,
 positiveIndex = 0, featureRows, inventedAtoms, sourceActivationRows = {},
 activationMatches, numerator, denominator, maximumNumerator = 0, maximumDenominator = 1},
 Do[eventIndex++;
 Do[positiveIndex++; AppendTo[positives, <|"EventIndex" -> positiveIndex, "TaskID" -> event["TaskID"],
 "Features" -> RelationalFeaturesS140[event["Models"], queryHash, event["PlanningDepth"]]]>],
 {queryHash, event["OptimalInputSHA256"]];
 Do[If[!MemberQ[event["OptimalInputSHA256"], queryHash], AppendTo[negatives,
 <|"EventIndex" -> eventIndex, "Features" -> RelationalFeaturesS140[event["Models"], queryHash,
 event["PlanningDepth"]]]>]],
 {queryHash, event["UnusedInputSHA256"]], {event, events}}];
 featureRows = Join[Lookup[positives, "Features"], Lookup[negatives, "Features"]];
 inventedAtoms = InventAtomsS140[featureRows];
 Do[covered = Select[positives, RuleMatchesS140[rule, #["Features"]] &];
 taskSupport = Length[DeleteDuplicates[Lookup[covered, "TaskID"]]];
 falseCount = Count[negatives, row_ /; RuleMatchesS140[rule, row["Features"]]];
 If[Length[covered] >= protocol["ConceptMinimumSupportEvents"] &&
 taskSupport >= protocol["ConceptMinimumDistinctSourceTasks"] && falseCount == 0,
 AppendTo[candidates, Join[rule, <|"CoveredEventIndices" -> Lookup[covered, "EventIndex"],
 "SupportEventCount" -> Length[covered], "DistinctSourceTaskSupportCount" -> taskSupport,

```

```

    "TrainingFalsePositiveCount" -> 0|>]], {rule, CandidateRulesS140[inventedAtoms]]};
uncovered = Range[Length[positives]]; valid = candidates;
While[Length[uncovered] > 0,
  ranked = Select[valid, Length[Intersection[#["CoveredEventIndices"], uncovered]] > 0 &];
  If[Length[ranked] === 0, Break[]];
  ranked = SortBy[ranked, {-Length[Intersection[#["CoveredEventIndices"], uncovered]],
    Length[#["Conditions"]], ToString[#["Conditions"], InputForm]} &];
  chosen = First[ranked]; body = <|"Conditions" -> chosen["Conditions"]|>;
  conceptID = "KC_" <> StringTake[IntegerString[Hash[chosen["Conditions"], "SHA256"], 16, 64], 16];
  AppendTo[selected, Join[<|"ConceptID" -> conceptID|>, body,
    KeyTake[chosen, {"SupportEventCount", "DistinctSourceTaskSupportCount", "TrainingFalsePositiveCount"}]]];
  uncovered = Complement[uncovered, chosen["CoveredEventIndices"]];
  valid = Select[valid, #["Conditions"] != chosen["Conditions"] &];
];
Do[
  activationMatches = Select[event["UnusedInputSHA256"], Function[queryHash,
    AnyTrue[selected, RuleMatchesS140[#,
      RelationalFeaturesS140[event["Models"], queryHash, event["PlanningDepth"]]] &]]];
  numerator = Length[activationMatches]; denominator = Max[1, Length[event["UnusedInputSHA256"]]];
  AppendTo[sourceActivationRows, <|"TaskID" -> event["TaskID"],
    "Numerator" -> numerator, "Denominator" -> denominator|>];
  If[numerator maximumDenominator > maximumNumerator denominator,
    maximumNumerator = numerator; maximumDenominator = denominator, {event, events}];
<|"LibraryType" -> "KERNEL_INVENTED_NUMERIC_RELATIONAL_PREDICATE_CONCEPTS",
  "InductionMethod" -> "DATA_DERIVED_THRESHOLD_ATOMS_PLUS_MDL_ZERO_FALSE_POSITIVE_SET_COVER",
  "PrimitiveNumericStateFeatures" -> {"KnownPrefixLength", "CoordinateCount",
    "RemainingCoordinateCount", "PlanningBudget", "CurrentDecisionClassCount"},
  "PrimitiveNumericQueryFeatures" -> {"WorstCaseKnownPrefixGain",
    "WorstCaseDecisionClassReduction", "WorstCaseRemainingDecisionClasses", "OutcomeBranchCount"},
  "PrimitiveComparisonOperators" -> {"Equal", "GreaterEqual", "LessEqual"},
  "PredeclaredNamedBooleanPredicates" -> {},
  "InventedAtomicPredicateCount" -> Length[inventedAtoms],
  "InventedAtomicPredicates" -> inventedAtoms,
  "SourceEventCount" -> Length[events], "SourceOptimalQueryExampleCount" -> Length[positives],
  "UnabstractedOptimalQueryExampleCount" -> Length[uncovered],
  "ConceptCount" -> Length[selected], "Concepts" -> selected,
  "RouterActivationCalibration" -> <|
    "Rule" -> "PREFERRED_FRACTION_NOT_ABOVE_MAXIMUM_SOURCE_EVENT_FRACTION",
    "MaximumSourcePreferredFractionNumerator" -> maximumNumerator,
    "MaximumSourcePreferredFractionDenominator" -> maximumDenominator,
    "SourceEventFractions" -> sourceActivationRows|>,
  "ConceptsMayPruneModels" -> False, "ConceptsMaySuppressFallbackQueries" -> False,
  "ExactDynamicProgrammingFallbackRequired" -> True|>
];
];

InstantiateConceptsS139[models_List, unused_List, planningDepth_Integer] := Module[{pairs = {}, matches},
  Do[matches = Select[Sort[unused], RuleMatchesS140[concept,
    RelationalFeaturesS140[models, #, planningDepth]] &];
  Do[If[!MemberQ[Lookup[pairs, "InputSHA256"], {}], queryHash], AppendTo[pairs,
    <|"ConceptID" -> concept["ConceptID"], "InputSHA256" -> queryHash|>]], {queryHash, matches}],
  {concept, conceptLibrary["Concepts"]}], pairs
];

ConceptPairsPassActivationEnvelopeS140[pairs_List, unused_List] :=
  Length[pairs] === 0 || !KeyExistsQ[conceptLibrary, "RouterActivationCalibration"] ||
  Length[pairs] conceptLibrary["RouterActivationCalibration"]["MaximumSourcePreferredFractionDenominator"] <=
  Length[unused] conceptLibrary["RouterActivationCalibration"]["MaximumSourcePreferredFractionNumerator"];

AdmittedConceptPairsS140[pairs_List, unused_List] :=
  If[ConceptPairsPassActivationEnvelopeS140[pairs, unused], pairs, {}];

PreferredQueriesG4[models_List, unused_List, planningDepth_Integer] := Module[{rawPairs, pairs},
  If[!TrueQ[$UseConceptsG4], Return[{}]];
  rawPairs = InstantiateConceptsS139[models, unused, planningDepth];
  pairs = AdmittedConceptPairsS140[rawPairs, unused];
  If[Length[rawPairs] > 0 && Length[pairs] === 0,
    IncrementCounterG4["ConceptActivationEnvelopeRejectedStateCount"];
    IncrementCounterG4["ConceptActivationEnvelopeRejectedQueryCount", Length[rawPairs]];
    IncrementCounterG4["ConceptNoCandidateStateCount"];
    Return[{}];
  If[Length[pairs] > 0, IncrementCounterG4["ConceptApplicableStateCount"];
    IncrementCounterG4["ConceptInstantiatedQueryCount", Length[pairs]],
    IncrementCounterG4["ConceptNoCandidateStateCount"]];

```

```

Lookup[pairs, "InputSHA256", {}]
];

SolveAtMostG4[models_List, unused_List, depth_Integer] := Module[
{key, ordered, preferred, branchRows, remaining, allSolvable, child, result},
If[DecisionCountG3[models] === 1, Return[<|"Solvable" -> True, "FirstInputSHA256" -> Null|>]];
If[depth === 0 || Length[unused] === 0, Return[UnsolvedG3[]]];
key = IntegerString[Hash[{Sort[Lookup[models, "ModelKey"]], Sort[unused], depth}, "SHA256"], 16, 64];
If[KeyExistsQ[$PlanMemoG3, key], Return[$PlanMemoG3[key]]];
IncrementCounterG4["ExpandedStateCount"];
preferred = PreferredQueriesG4[models, Sort[unused], depth];
ordered = Join[preferred, Select[Sort[unused], !MemberQ[preferred, #] &]];
result = Catch[
Do[
IncrementCounterG4["QueryEvaluationCount"];
branchRows = BranchesG3[models, queryHash];
IncrementCounterG4["OutcomeBranchEvaluationCount", Length[branchRows]];
remaining = DeleteCases[unused, queryHash]; allSolvable = True;
Do[child = SolveAtMostG4[branch, remaining, depth - 1];
If[!TrueQ[child["Solvable"]], allSolvable = False; Break[]], {branch, branchRows}];
If[TrueQ[allSolvable],
Throw[<|"Solvable" -> True, "FirstInputSHA256" -> queryHash|>, "PlanFound"]];
If[MemberQ[preferred, queryHash], IncrementCounterG4["ConceptPreferredQueryRejectedCount"],
{queryHash, ordered}];
UnsolvedG3[],
"PlanFound"
];
AssociateTo[$PlanMemoG3, key -> result]; result
];

FindMinimalPlanG4[models_List, unused_List, maximumDepth_Integer, queryByHash_Association,
useConcepts_] := Module[{found, plan},
$PlanMemoG3 = <|>; ResetPlanCountersG4[]; $QueryByHashG4 = queryByHash;
$UseConceptsG4 = TrueQ[useConcepts]; found = UnsolvedG3[];
Do[plan = SolveAtMostG4[models, unused, depth]; If[TrueQ[plan["Solvable"]],
found = Join[plan, <|"RequiredDepth" -> depth|>]; Break[]], {depth, 0, maximumDepth}];
Join[found, <|"WorkCounters" -> $PlanCountersG4|>]
];

OptimalInputsS139[models_List, unused_List, depth_Integer, queryByHash_Association] := Module[
{optimal = {}, remaining, branchRows, allSolvable, child},
$UseConceptsG4 = False; $QueryByHashG4 = queryByHash;
Do[remaining = DeleteCases[unused, queryHash]; branchRows = BranchesG3[models, queryHash];
allSolvable = True;
Do[child = SolveAtMostG4[branch, remaining, depth - 1];
If[!TrueQ[child["Solvable"]], allSolvable = False; Break[]], {branch, branchRows}];
If[TrueQ[allSolvable], AppendTo[optimal, queryHash], {queryHash, Sort[unused]}];
optimal
];

RunOracleG3[mode_String, taskID_String : "NONE", queryNumber_String : "NONE"] := Module[{command},
command = "set TCCT_ORACLE_MODE=" <> mode <> "&set TCCT_TASK=" <> taskID <>
"&set TCCT_QUERY=" <> queryNumber <> "&" <> protocol["PowerShellEngine"] <>
" -NoProfile -NonInteractive -EncodedCommand " <> protocol["OracleBridgeEncodedCommand"];
Run[command]
];

CallOracleG3[taskID_String, queryNumber_String, query_Association] := Module[{request, response, exitCode},
request = <|"ProtocolSHA256" -> protocolHash, "TaskID" -> taskID, "QueryNumber" -> queryNumber,
"Input" -> query["Input"], "InputSHA256" -> query["InputSHA256"],
"GeneratedByTCCTKernel" -> True, "TestOutputAccessed" -> False,
"HiddenProgramAccessedByLearner" -> False|>;
Export[oracleRequestPath, request, "RawJSON", "Compact" -> False];
exitCode = RunOracleG3["query", taskID, queryNumber];
If[exitCode != 0, Print["FATAL: oracle query failed"]; Exit[20]];
response = Import[oracleResponsePath, "RawJSON"];
If[response["ProtocolSHA256"] != protocolHash || response["TaskID"] != taskID ||
response["QueryNumber"] != queryNumber || response["Input"] != query["Input"] ||
response["InputSHA256"] != query["InputSHA256"], Print["FATAL: invalid oracle response"]; Exit[21]];
response
];

$LearningEventsS139 = {};

```

```

LearnSourceTaskS139[task_Association] := Module[
{queries, queryByHash, models, initialModels, unused, trace = {}, plan, remainingCap,
query, queryNumber, response, keep, committed, initialDepth},
queries = BuildQueriesG3[task]; queryByHash = AssociationThread[Lookup[queries, "InputSHA256"] -> queries];
models = BuildModelsG3[task, queries]; initialModels = models; unused = Lookup[queries, "InputSHA256"];
plan = FindMinimalPlanG4[models, unused, protocol["MaximumPlanningDepth"], queryByHash, False];
initialDepth = If[TrueQ[plan["Solvable"]], plan["RequiredDepth"], Null];
While[DecisionCountG3[models] > 1 && Length[trace] < protocol["MaximumActiveQueriesPerTask"],
remainingCap = protocol["MaximumActiveQueriesPerTask"] - Length[trace];
If[Length[trace] > 0, plan = FindMinimalPlanG4[models, unused, remainingCap, queryByHash, False]];
If[!TrueQ[plan["Solvable"]] || plan["RequiredDepth"] == 0, Break[]];
query = queryByHash[plan["FirstInputSHA256"]];
AppendTo[$LearningEventsS139, <|"TaskID" -> task["TaskID"], "Models" -> models,
"UnusedInputSHA256" -> unused,
"PlanningDepth" -> plan["RequiredDepth"],
"OptimalInputSHA256" -> OptimalInputsS139[models, unused, plan["RequiredDepth"], queryByHash]|>];
queryNumber = "KQ" <> IntegerString[Length[trace] + 1, 10, 2];
response = CallOracleG3[task["TaskID"], queryNumber, query];
keep = PredictionG3[#, query["InputSHA256"]] == response["Output"] & /@ models;
AppendTo[trace, <|"QueryNumber" -> queryNumber, "InputSHA256" -> query["InputSHA256"],
"QueryKind" -> query["Kind"], "QueryLevel" -> query["Level"], "QueryPrefix" -> query["Prefix"],
"DecisionClassCountBefore" -> DecisionCountG3[models],
"DecisionClassCountAfter" -> DecisionCountG3[Pick[models, keep, True]],
"GeneratedByTCCTKernel" -> True|>];
models = Pick[models, keep, True]; unused = DeleteCases[unused, query["InputSHA256"]];
];
committed = DecisionCountG3[models] == 1;
<|"TaskID" -> task["TaskID"], "InitialSemanticClassCount" -> Length[initialModels],
"InitialCertifiedMinimumDepth" -> initialDepth, "ActiveQueryCount" -> Length[trace],
"ActiveQueryTrace" -> trace, "DecisionCertified" -> committed,
"ConceptLabelsAccessed" -> False, "ConceptBodiesAccessed" -> False|>
];

EvaluateTaskG3[task_Association] := Module[
{queries, queryByHash, models, initialModels, unused, trace = {}, initialGuided, initialBaseline,
guided, baseline, remainingCap, query, response, keep, queryNumber, stopReason, committed,
runtime, value, rawRootPairs, rootPairs, guidedTotal = 0, baselineTotal = 0,
guidedStates = 0, baselineStates = 0, parity = True},
queries = BuildQueriesG3[task]; queryByHash = AssociationThread[Lookup[queries, "InputSHA256"] -> queries];
models = BuildModelsG3[task, queries]; initialModels = models; unused = Lookup[queries, "InputSHA256"];
initialGuided = FindMinimalPlanG4[models, unused, protocol["MaximumPlanningDepth"], queryByHash, True];
initialBaseline = FindMinimalPlanG4[models, unused, protocol["MaximumPlanningDepth"], queryByHash, False];
parity = initialGuided["Solvable"] == initialBaseline["Solvable"] &&
initialGuided["RequiredDepth"] == initialBaseline["RequiredDepth"];
guidedTotal += initialGuided["WorkCounters"]["QueryEvaluationCount"];
baselineTotal += initialBaseline["WorkCounters"]["QueryEvaluationCount"];
guidedStates += initialGuided["WorkCounters"]["ExpandedStateCount"];
baselineStates += initialBaseline["WorkCounters"]["ExpandedStateCount"];
{runtime, value} = AbsoluteTiming[While[DecisionCountG3[models] > 1 && Length[trace] < protocol["MaximumActiveQueriesPerTask"],
remainingCap = Min[protocol["MaximumPlanningDepth"], protocol["MaximumActiveQueriesPerTask"] - Length[trace]];
If[Length[trace] == 0, guided = initialGuided; baseline = initialBaseline,
guided = FindMinimalPlanG4[models, unused, remainingCap, queryByHash, True];
baseline = FindMinimalPlanG4[models, unused, remainingCap, queryByHash, False];
parity = parity && guided["Solvable"] == baseline["Solvable"] &&
guided["RequiredDepth"] == baseline["RequiredDepth"];
guidedTotal += guided["WorkCounters"]["QueryEvaluationCount"];
baselineTotal += baseline["WorkCounters"]["QueryEvaluationCount"];
guidedStates += guided["WorkCounters"]["ExpandedStateCount"];
baselineStates += baseline["WorkCounters"]["ExpandedStateCount"]];
If[!TrueQ[guided["Solvable"]] || guided["RequiredDepth"] == 0,
stopReason = "NO_PLAN_WITHIN_RESOURCE_DEPTH"; Break[]];
rawRootPairs = InstantiateConceptsS139[models, unused, guided["RequiredDepth"]];
rootPairs = AdmittedConceptPairsS140[rawRootPairs, unused];
query = queryByHash[guided["FirstInputSHA256"]];
queryNumber = "KQ" <> IntegerString[Length[trace] + 1, 10, 2];
response = CallOracleG3[task["TaskID"], queryNumber, query];
keep = PredictionG3[#, query["InputSHA256"]] == response["Output"] & /@ models;
AppendTo[trace, <|"QueryNumber" -> queryNumber, "Input" -> query["Input"],
"InputSHA256" -> query["InputSHA256"], "QueryKind" -> query["Kind"],
"QueryLevel" -> query["Level"], "QueryPrefix" -> query["Prefix"],
"AdmissionMode" -> "CONCEPT_GUIDED_EXACT_DP_WITH_FALLBACK",
"CertifiedMinimumDepthBefore" -> guided["RequiredDepth"],
"BaselineCertifiedMinimumDepthBefore" -> baseline["RequiredDepth"],
];

```

```

"BaselineFirstInputSHA256" -> baseline["FirstInputSHA256"],
"RootConceptRawMatched" -> (Length[rawRootPairs] > 0),
"RootConceptActivationEnvelopeRejected" ->
  (Length[rawRootPairs] > 0 && Length[rootPairs] == 0),
"RootConceptMatched" -> (Length[rootPairs] > 0),
"RootConceptIDs" -> DeleteDuplicates[Lookup[rootPairs, "ConceptID", {}]],
"RootConceptPreferredInputSHA256" -> Lookup[rootPairs, "InputSHA256", {}],
"SelectedQueryWasRootConceptPreference" -> MemberQ[Lookup[rootPairs, "InputSHA256", {}], query["InputSHA256"]],
"RootConceptFallbackUsed" -> (Length[rootPairs] == 0),
"GuidedWorkCounters" -> guided["WorkCounters"],
"BaselineWorkCounters" -> baseline["WorkCounters"],
"DecisionClassCountBefore" -> DecisionCountG3[models], "SemanticClassCountBefore" -> Length[models],
"OracleOutput" -> response["Output"], "DecisionClassCountAfter" -> DecisionCountG3[Pick[models, keep, True]],
"SemanticClassCountAfter" -> Length[Pick[models, keep, True]], "GeneratedByTCCTKernel" -> True,
"TestOutputAccessed" -> False, "HiddenProgramAccessedByLearner" -> False];
models = Pick[models, keep, True]; unused = DeleteCases[unused, query["InputSHA256"]];
];
committed = DecisionCountG3[models] == 1;
If[!StringQ[stopReason], stopReason = If[committed, "DECISION_CERTIFIED", "NO_PLAN_WITHIN_RESOURCE_DEPTH"]];
<|"TaskID" -> task["TaskID"], "RuntimeSeconds" -> runtime,
  "InitialSemanticClassCount" -> Length[initialModels], "InitialDecisionClassCount" -> DecisionCountG3[initialModels],
  "InitialPlanExistsWithinCap" -> TrueQ[initialGuided["Solvable"]],
  "InitialCertifiedMinimumDepth" -> If[TrueQ[initialGuided["Solvable"]], initialGuided["RequiredDepth"], Null],
  "InitialBaselineCertifiedMinimumDepth" -> If[TrueQ[initialBaseline["Solvable"]], initialBaseline["RequiredDepth"], Null],
  "PairedDepthParity" -> parity,
  "InitialGuidedWorkCounters" -> initialGuided["WorkCounters"],
  "InitialBaselineWorkCounters" -> initialBaseline["WorkCounters"],
  "GuidedQueryEvaluationCount" -> guidedTotal,
  "BaselineQueryEvaluationCount" -> baselineTotal,
  "GuidedExpandedStateCount" -> guidedStates,
  "BaselineExpandedStateCount" -> baselineStates,
  "FinalSemanticClassCount" -> Length[models], "FinalDecisionClassCount" -> DecisionCountG3[models],
  "ActiveQueryCount" -> Length[trace], "ActiveQueryTrace" -> trace, "AdaptiveStopReason" -> stopReason,
  "DecisionCertified" -> committed, "TestPredictionCommitted" -> committed,
  "CommittedTestPrediction" -> If[committed, First[models]["TestPrediction"], Null],
  "Status" -> If[committed, "DECISION_CERTIFIED", "NO_PLAN_WITHIN_RESOURCE_DEPTH"],
  "TestOutputAccessed" -> False, "HiddenProgramAccessedByLearner" -> False];
];

Print[protocol["Stage"]];
Print["Native kernel forms anonymous relational planning concepts before opening target tasks"];
resetCode = RunOracleG3["reset"];
If[resetCode != 0, Print["FATAL: oracle reset failed"]; Exit[22]];
{sourceRuntime, sourceRows} = AbsoluteTiming[LearnSourceTaskS139 /@ public["SourceTasks"]];
sourceDepths = Lookup[sourceRows, "InitialCertifiedMinimumDepth"];
sourceQueryCount = Total[Lookup[sourceRows, "ActiveQueryCount"]];
conceptLibrary = InduceConceptsS139[$LearningEventsS139];
conceptLibrary = Join[conceptLibrary, <|"CreatedByNativeWolframKernel" -> True,
  "CreatedAfterSourceQueryCount" -> sourceQueryCount, "CreatedBeforeTargetTaskExecution" -> True,
  "GeneratorProvidedConceptLabelCount" -> 0, "GeneratorProvidedConceptBodyCount" -> 0|>];
Export[conceptLibraryPath, conceptLibrary, "RawJSON", "Compact" -> False];
conceptLibraryHash = IntegerString[FileHash[conceptLibraryPath, "SHA256"], 16, 64];
Print["source depths=", sourceDepths, " events=", Length[$LearningEventsS139],
  " concepts=", conceptLibrary["ConceptCount"]];
{targetRuntime, taskRows} = AbsoluteTiming[EvaluateTaskG3 /@ public["TargetTasks"]];
Do[Print[Row["TaskID", " depth=", row["InitialCertifiedMinimumDepth"], " queries=",
  row["ActiveQueryCount"], " status=", row["Status"]], {row, taskRows}];
initialDepths = Lookup[taskRows, "InitialCertifiedMinimumDepth"];
tracePolicyPass = And @@ Flatten[Map[Function[row, Map[Function[q,
  q["AdmissionMode"] == "CONCEPT_GUIDED_EXACT_DP_WITH_FALLBACK" &&
  IntegerQ[q["CertifiedMinimumDepthBefore"]] && q["CertifiedMinimumDepthBefore"] >= 1 &&
  q["BaselineCertifiedMinimumDepthBefore"] == q["CertifiedMinimumDepthBefore"]],
  row["ActiveQueryTrace"]]], taskRows];
commitCount = Count[Lookup[taskRows, "TestPredictionCommitted"], True];
oracleLogCount = Length@Select[StringSplit[Import[oracleLogPath, "Text"], "\n"], StringLength[#] > 0 &];
guidedQueryEvaluations = Total[Lookup[taskRows, "GuidedQueryEvaluationCount"]];
baselineQueryEvaluations = Total[Lookup[taskRows, "BaselineQueryEvaluationCount"]];
pairedDepthParity = And @@ Lookup[taskRows, "PairedDepthParity"];
rootConceptUseCount = Count[Flatten[Lookup[taskRows, "ActiveQueryTrace", 1], q_ /;
  TrueQ[q["SelectedQueryWasRootConceptPreference"]]];
rootFallbackCount = Count[Flatten[Lookup[taskRows, "ActiveQueryTrace", 1], q_ /;
  TrueQ[q["RootConceptFallbackUsed"]]];
preferredRejectionCount = Total[Lookup[taskRows, "InitialGuidedWorkCounters"],

```



```

"ConceptPreferredQueryRejectedCount"]];
conceptSupportPass = conceptLibrary["ConceptCount"] >= 1 && And @@
  (#["SupportEventCount"] >= 2 && #["DistinctSourceTaskSupportCount"] >= 2 &&
    #["TrainingFalsePositiveCount"] === 0 & /@ conceptLibrary["Concepts"]);
preScorePass = sourceDepths === {3, 2, 1} && sourceQueryCount >= 3 &&
  Length[$LearningEventsS139] >= 3 && conceptSupportPass &&
  Sort[initialDepths] === {1, 2, 2, 2, 3} && tracePolicyPass && commitCount === 5 &&
  pairedDepthParity && guidedQueryEvaluations < baselineQueryEvaluations &&
  rootConceptUseCount >= 1 && rootFallbackCount >= 1 &&
  oracleLogCount === sourceQueryCount + Total[Lookup[taskRows, "ActiveQueryCount"]] &&
  !MemberQ[Lookup[taskRows, "TestOutputAccessed"], True] &&
  !MemberQ[Lookup[taskRows, "HiddenProgramAccessedByLearner"], True];
result = <|"Stage" -> protocol["Stage"], "EvidenceStatus" -> protocol["EvidenceStatus"],
  "ProtocolSHA256" -> protocolHash, "WolframRunnerSHA256" -> runnerHash,
  "OracleResponderSHA256" -> oracleHash, "KernelInducedConceptLibrarySHA256" -> conceptLibraryHash,
  "NativeWolframExecution" -> True, "WolframVersion" -> $Version,
  "RuntimeSeconds" -> sourceRuntime + targetRuntime, "SourceTaskResults" -> sourceRows,
  "SourcePlanningEventCount" -> Length[$LearningEventsS139],
  "KernelInducedConceptLibrary" -> conceptLibrary, "TargetTaskResults" -> taskRows,
  "ObservedTargetActiveQueryCounts" -> Lookup[taskRows, "ActiveQueryCount"],
  "ObservedTargetCertifiedDepths" -> initialDepths,
  "AggregateGuidedQueryEvaluationCount" -> guidedQueryEvaluations,
  "AggregateBaselineQueryEvaluationCount" -> baselineQueryEvaluations,
  "DeterministicPlanningWorkReduced" -> (guidedQueryEvaluations < baselineQueryEvaluations),
  "PairedDepthParity" -> pairedDepthParity, "RootConceptSelectedQueryCount" -> rootConceptUseCount,
  "RootConceptFallbackCount" -> rootFallbackCount,
  "ConceptPreferredQueryRejectedCount" -> preferredRejectionCount,
  "OracleQueryLogLineCount" -> oracleLogCount, "TracePolicyPreScorePass" -> tracePolicyPass,
  "ConceptBodiesCreatedByNativeKernel" -> True, "ConceptBodiesProvidedByGenerator" -> False,
  "NativePreScorePass" -> preScorePass, "CoreRewriteFreezeDedupModified" -> False,
  "Conclusion" -> If[preScorePass,
    "KERNEL_NATIVE_CONCEPT_FORMATION_COMPLETE_AWAITING_SEALED_SCORE", "NATIVE_PREScore_FAILURE"]]>;
Export[resultPath, result, "RawJSON", "Compact" -> False];
Print["native pre-score pass=", preScorePass, " runtime=", sourceRuntime + targetRuntime];
Exit[0];

```

Native Wolfram execution result

```
{
  "Stage": "S140-C DIAGNOSTIC R1 source-selectivity activation envelope replay",
  "EvidenceStatus": "RETROSPECTIVE_REPLAY_OF_FROZEN_S140B_HIGH_PREFIX_FAILURE",
  "ProtocolSHA256": "6ef80df675aeda0f9b5c033499a9abd1663cf9777331a54e2741ab12fec3b83",
  "WolframRunnerSHA256": "de98b473aec8f54dc1a0a8362c0df507e38564f46a96b980e5567325e9136ea2",
  "OracleResponderSHA256": "c3ee6cdbf12b6bb82938b5b5cdc34dee2d6fbedad8caff7aa40a6f26f5a686f7",
  "KernelInducedConceptLibrarySHA256": "5710624dc33a049149752eebf49c566ecff2444e54b23ca0e09992ee4a39e9",
  "NativeWolframExecution": true,
  "WolframVersion": "15.0.0 for Microsoft Windows (64-bit) (May 26, 2026)",
  "RuntimeSeconds": 41.3265465,
  "SourceTaskResults": [
    {
      "TaskID": "SRC001",
      "InitialSemanticClassCount": 24,
      "InitialCertifiedMinimumDepth": 3,
      "ActiveQueryCount": 1,
      "ActiveQueryTrace": [
        {
          "QueryNumber": "KQ01",
          "InputSHA256": "15288af76cc3cbda7aef8c63abe6534372ee63444d7af11a6d18b280e1b50d92",
          "QueryKind": "DECISION",
          "QueryLevel": null,
          "QueryPrefix": [
            1,
            0,
            1,
            1
          ],
          "DecisionClassCountBefore": 3,
          "DecisionClassCountAfter": 1,
          "GeneratedByTCCTKernel": true
        }
      ],
      "DecisionCertified": true,
      "ConceptLabelsAccessed": false,
      "ConceptBodiesAccessed": false
    },
    {
      "TaskID": "SRC002",
      "InitialSemanticClassCount": 12,
      "InitialCertifiedMinimumDepth": 2,
      "ActiveQueryCount": 2,
      "ActiveQueryTrace": [
        {
          "QueryNumber": "KQ01",
          "InputSHA256": "30ef1006351c773a98c80a99d110aa2d190a09bc8d40d7eaf2139a9cd80eb604",
          "QueryKind": "CALIBRATE",
          "QueryLevel": 3,
          "QueryPrefix": [
            1,
            1,
            0
          ],
          "DecisionClassCountBefore": 3,
          "DecisionClassCountAfter": 3,
          "GeneratedByTCCTKernel": true
        },
        {
          "QueryNumber": "KQ02",
          "InputSHA256": "82feafa2ab34cb2649a2819b8691699ad25c06cce2b0d26ade34239da8631cd4",
          "QueryKind": "DECISION",
          "QueryLevel": null,
          "QueryPrefix": [
            1,
            1,
            0,
            0
          ]
        }
      ]
    }
  ]
}
```

```

        "DecisionClassCountBefore": 3,
        "DecisionClassCountAfter": 1,
        "GeneratedByTCCTKernel": true
    }
},
"DecisionCertified": true,
"ConceptLabelsAccessed": false,
"ConceptBodiesAccessed": false
},
{
    "TaskID": "SRC003",
    "InitialSemanticClassCount": 6,
    "InitialCertifiedMinimumDepth": 1,
    "ActiveQueryCount": 1,
    "ActiveQueryTrace": [
        {
            "QueryNumber": "KQ01",
            "InputSHA256": "937d0f866b239d9fa589afc1388755345b71d31239f8680d3f1f3ec22e9c6190",
            "QueryKind": "DECISION",
            "QueryLevel": null,
            "QueryPrefix": [
                0,
                0,
                0
            ],
            "DecisionClassCountBefore": 3,
            "DecisionClassCountAfter": 1,
            "GeneratedByTCCTKernel": true
        }
    ],
    "DecisionCertified": true,
    "ConceptLabelsAccessed": false,
    "ConceptBodiesAccessed": false
}
],
"SourcePlanningEventCount": 4,
"KernelInducedConceptLibrary": {
    "LibraryType": "KERNEL_INVENTED_NUMERIC_RELATIONAL_PREDICATE_CONCEPTS",
    "InductionMethod": "DATA_DERIVED_THRESHOLD_ATOMS_PLUS_MDL_ZERO_FALSE_POSITIVE_SET_COVER",
    "PrimitiveNumericStateFeatures": [
        "KnownPrefixLength",
        "CoordinateCount",
        "RemainingCoordinateCount",
        "PlanningBudget",
        "CurrentDecisionClassCount"
    ],
    "PrimitiveNumericQueryFeatures": [
        "WorstCaseKnownPrefixGain",
        "WorstCaseDecisionClassReduction",
        "WorstCaseRemainingDecisionClasses",
        "OutcomeBranchCount"
    ],
    "PrimitiveComparisonOperators": [
        "Equal",
        "GreaterEqual",
        "LessEqual"
    ],
    "PredeclaredNamedBooleanPredicates": [],
    "InventedAtomicPredicateCount": 63,
    "InventedAtomicPredicates": [
        [
            "KnownPrefixLength",
            "Equal",
            2
        ],
        [
            "KnownPrefixLength",
            "GreaterEqual",
            2
        ],
        [
            "KnownPrefixLength",
            "LessEqual",
            2
        ]
    ]
}

```

```

2
],
[
  "KnownPrefixLength",
  "Equal",
  3
],
[
  "KnownPrefixLength",
  "GreaterEqual",
  3
],
[
  "KnownPrefixLength",
  "LessEqual",
  3
],
[
  "KnownPrefixLength",
  "Equal",
  4
],
[
  "KnownPrefixLength",
  "GreaterEqual",
  4
],
[
  "KnownPrefixLength",
  "LessEqual",
  4
],
[
  "CoordinateCount",
  "Equal",
  3
],
[
  "CoordinateCount",
  "GreaterEqual",
  3
],
[
  "CoordinateCount",
  "LessEqual",
  3
],
[
  "CoordinateCount",
  "Equal",
  4
],
[
  "CoordinateCount",
  "GreaterEqual",
  4
],
[
  "CoordinateCount",
  "LessEqual",
  4
],
[
  "RemainingCoordinateCount",
  "Equal",
  0
],
[
  "RemainingCoordinateCount",
  "GreaterEqual",
  0
],
[

```

```

    "RemainingCoordinateCount",
    "LessEqual",
    0
  ],
  [
    "RemainingCoordinateCount",
    "Equal",
    1
  ],
  [
    "RemainingCoordinateCount",
    "GreaterEqual",
    1
  ],
  [
    "RemainingCoordinateCount",
    "LessEqual",
    1
  ],
  [
    "RemainingCoordinateCount",
    "Equal",
    2
  ],
  [
    "RemainingCoordinateCount",
    "GreaterEqual",
    2
  ],
  [
    "RemainingCoordinateCount",
    "LessEqual",
    2
  ],
  [
    "PlanningBudget",
    "Equal",
    1
  ],
  [
    "PlanningBudget",
    "GreaterEqual",
    1
  ],
  [
    "PlanningBudget",
    "LessEqual",
    1
  ],
  [
    "PlanningBudget",
    "Equal",
    2
  ],
  [
    "PlanningBudget",
    "GreaterEqual",
    2
  ],
  [
    "PlanningBudget",
    "LessEqual",
    2
  ],
  [
    "PlanningBudget",
    "Equal",
    3
  ],
  [
    "PlanningBudget",
    "GreaterEqual",
    3
  ]

```

```

],
[
  "PlanningBudget",
  "LessEqual",
  3
],
[
  "CurrentDecisionClassCount",
  "Equal",
  3
],
[
  "CurrentDecisionClassCount",
  "GreaterEqual",
  3
],
[
  "CurrentDecisionClassCount",
  "LessEqual",
  3
],
[
  "WorstCaseKnownPrefixGain",
  "Equal",
  0
],
[
  "WorstCaseKnownPrefixGain",
  "GreaterEqual",
  0
],
[
  "WorstCaseKnownPrefixGain",
  "LessEqual",
  0
],
[
  "WorstCaseKnownPrefixGain",
  "Equal",
  1
],
[
  "WorstCaseKnownPrefixGain",
  "GreaterEqual",
  1
],
[
  "WorstCaseKnownPrefixGain",
  "LessEqual",
  1
],
[
  "WorstCaseDecisionClassReduction",
  "Equal",
  0
],
[
  "WorstCaseDecisionClassReduction",
  "GreaterEqual",
  0
],
[
  "WorstCaseDecisionClassReduction",
  "LessEqual",
  0
],
[
  "WorstCaseDecisionClassReduction",
  "Equal",
  2
],
[
  "WorstCaseDecisionClassReduction",

```

```

    "GreaterEqual",
    2
  ],
  [
    "WorstCaseDecisionClassReduction",
    "LessEqual",
    2
  ],
  [
    "WorstCaseRemainingDecisionClasses",
    "Equal",
    1
  ],
  [
    "WorstCaseRemainingDecisionClasses",
    "GreaterEqual",
    1
  ],
  [
    "WorstCaseRemainingDecisionClasses",
    "LessEqual",
    1
  ],
  [
    "WorstCaseRemainingDecisionClasses",
    "Equal",
    3
  ],
  [
    "WorstCaseRemainingDecisionClasses",
    "GreaterEqual",
    3
  ],
  [
    "WorstCaseRemainingDecisionClasses",
    "LessEqual",
    3
  ],
  [
    "OutcomeBranchCount",
    "Equal",
    1
  ],
  [
    "OutcomeBranchCount",
    "GreaterEqual",
    1
  ],
  [
    "OutcomeBranchCount",
    "LessEqual",
    1
  ],
  [
    "OutcomeBranchCount",
    "Equal",
    2
  ],
  [
    "OutcomeBranchCount",
    "GreaterEqual",
    2
  ],
  [
    "OutcomeBranchCount",
    "LessEqual",
    2
  ],
  [
    "OutcomeBranchCount",
    "Equal",
    3
  ],
],

```

```

[
  "OutcomeBranchCount",
  "GreaterEqual",
  3
],
[
  "OutcomeBranchCount",
  "LessEqual",
  3
]
],
"SourceEventCount": 4,
"SourceOptimalQueryExampleCount": 12,
"UnabstractedOptimalQueryExampleCount": 0,
"ConceptCount": 2,
"Concepts": [
  {
    "ConceptID": "KC_f5985bcf11108178",
    "Conditions": [
      [
        "OutcomeBranchCount",
        "Equal",
        3
      ]
    ],
    "SupportEventCount": 10,
    "DistinctSourceTaskSupportCount": 3,
    "TrainingFalsePositiveCount": 0
  },
  {
    "ConceptID": "KC_ed4b21f1f97412bc",
    "Conditions": [
      [
        "WorstCaseKnownPrefixGain",
        "Equal",
        1
      ]
    ],
    "SupportEventCount": 4,
    "DistinctSourceTaskSupportCount": 2,
    "TrainingFalsePositiveCount": 0
  }
],
"RouterActivationCalibration": {
  "Rule": "PREFERRED_FRACTION_NOT_ABOVE_MAXIMUM_SOURCE_EVENT_FRACTION",
  "MaximumSourcePreferredFractionNumerator": 7,
  "MaximumSourcePreferredFractionDenominator": 30,
  "SourceEventFractions": [
    {
      "TaskID": "SRC001",
      "Numerator": 7,
      "Denominator": 30
    },
    {
      "TaskID": "SRC002",
      "Numerator": 3,
      "Denominator": 43
    },
    {
      "TaskID": "SRC002",
      "Numerator": 1,
      "Denominator": 42
    },
    {
      "TaskID": "SRC003",
      "Numerator": 1,
      "Denominator": 20
    }
  ]
},
"ConceptsMayPruneModels": false,
"ConceptsMaySuppressFallbackQueries": false,
"ExactDynamicProgrammingFallbackRequired": true,

```


[illegible]

```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
]
```

```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
2,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
2,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
```

```
0,
0,
0,
0,
2,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
]
```



```

"BaselineWorkCounters": {
  "ExpandedStateCount": 4,
  "QueryEvaluationCount": 9,
  "OutcomeBranchEvaluationCount": 24,
  "ConceptApplicableStateCount": 0,
  "ConceptInstantiatedQueryCount": 0,
  "ConceptNoCandidateStateCount": 0,
  "ConceptPreferredQueryRejectedCount": 0,
  "ConceptActivationEnvelopeRejectedStateCount": 0,
  "ConceptActivationEnvelopeRejectedQueryCount": 0
},
"DecisionClassCountBefore": 3,
"SemanticClassCountBefore": 12,
"OracleOutput": [
  [
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    8
  ],
  [
    0,
    0,
    8,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0
  ],
  [
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0
  ],
  [
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0
  ]
]

```



```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
6,
6,
0,
0,
0,
0,
```

[illegible]


```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
```



```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
6,
```


[illegible]

```
8
],
[
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
]
```

[illegible]

```
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  2,
  2,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  2,
  2,
  0,
  0,
  0,
  0,
],
[
```


0,
0,
0,
0,
0,
0,
0,
5[illegible][illegible][illegible]
$$\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \\ \theta_7 \\ \theta_8 \end{bmatrix},$$

0,
0,
0,
0,
0,
0,
0)[illegible][illegible][illegible][illegible]


```
[
  0,
  0,
  0,
  0,
  0,
  0,
  8,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  8,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
]
```


[illegible]

[illegible]

```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
```


$$\begin{pmatrix} \theta_0 \\ \vdots \\ \theta_{n-1} \end{pmatrix}, \quad \begin{pmatrix} \theta_0 \\ \vdots \\ \theta_{n-1} \end{pmatrix}$$

[illegible]


```
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0
```



```
0,  
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0
```

```

0,
0,
0,
0,
9,
9,
9,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
7,
7,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
6,
6,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
]
],
"Status": "DECISION_CERTIFIED",
"TestOutputAccessed": false,
"HiddenProgramAccessedByLearner": false
},
{
"TaskID": "KT003",
"RuntimeSeconds": 8.5724033,
"InitialSemanticClassCount": 36,
"InitialDecisionClassCount": 3,
"InitialPlanExistsWithinCap": true,
"InitialCertifiedMinimumDepth": 3,
"InitialBaselineCertifiedMinimumDepth": 3,
"PairedDepthParity": true,
"InitialGuidedWorkCounters": {
"ExpandedStateCount": 54,
"QueryEvaluationCount": 1604,
"OutcomeBranchEvaluationCount": 2408,
"ConceptApplicableStateCount": 22,
"ConceptInstantiatedQueryCount": 82,

```



```
0
],
[
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
]
```



```
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
  0,
],
[
```



```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
```

$$\begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \\ \theta_7 \\ \theta_8 \\ \theta_9 \\ \theta_{10} \\ \theta_{11} \\ \theta_{12} \\ \theta_{13} \\ \theta_{14} \\ \theta_{15} \\ \theta_{16} \\ \theta_{17} \\ \theta_{18} \\ \theta_{19} \\ \theta_{20} \\ \theta_{21} \\ \theta_{22} \\ \theta_{23} \\ \theta_{24} \\ \theta_{25} \\ \theta_{26} \\ \theta_{27} \\ \theta_{28} \\ \theta_{29} \\ \theta_{30} \\ \theta_{31} \\ \theta_{32} \\ \theta_{33} \\ \theta_{34} \\ \theta_{35} \\ \theta_{36} \\ \theta_{37} \\ \theta_{38} \\ \theta_{39} \\ \theta_{40} \\ \theta_{41} \\ \theta_{42} \\ \theta_{43} \\ \theta_{44} \\ \theta_{45} \\ \theta_{46} \\ \theta_{47} \\ \theta_{48} \\ \theta_{49} \\ \theta_{50} \\ \theta_{51} \\ \theta_{52} \\ \theta_{53} \\ \theta_{54} \\ \theta_{55} \\ \theta_{56} \\ \theta_{57} \\ \theta_{58} \\ \theta_{59} \\ \theta_{60} \\ \theta_{61} \\ \theta_{62} \\ \theta_{63} \\ \theta_{64} \\ \theta_{65} \\ \theta_{66} \\ \theta_{67} \\ \theta_{68} \\ \theta_{69} \\ \theta_{70} \\ \theta_{71} \\ \theta_{72} \\ \theta_{73} \\ \theta_{74} \\ \theta_{75} \\ \theta_{76} \\ \theta_{77} \\ \theta_{78} \\ \theta_{79} \\ \theta_{80} \\ \theta_{81} \\ \theta_{82} \\ \theta_{83} \\ \theta_{84} \\ \theta_{85} \\ \theta_{86} \\ \theta_{87} \\ \theta_{88} \\ \theta_{89} \\ \theta_{90} \\ \theta_{91} \\ \theta_{92} \\ \theta_{93} \\ \theta_{94} \\ \theta_{95} \\ \theta_{96} \\ \theta_{97} \\ \theta_{98} \\ \theta_{99} \\ \theta_{100} \end{bmatrix}$$

[illegible]

[illegible]


```

0,
0,
0,
0,
0,
0,
0,
0,
0,
0
]
],
"DecisionClassCountAfter": 3,
"SemanticClassCountAfter": 18,
"GeneratedByTCCTKernel": true,
"TestOutputAccessed": false,
"HiddenProgramAccessedByLearner": false
},
{
  "QueryNumber": "KQ02",
  "Input": [
    [
      0,
      0,
      0,
      9,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0
    ],
    [
      0,
      0,
      0,
      8,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0,
      0
    ]
  ],
  [
    [
      0,
      0,
      0,
      0,

```


[illegible]


```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
8,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
```

[illegible]

[illegible]

[illegible]


```
0,
0,
6,
6,
6,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
9,
9,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
7,
7,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
6,
6,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
```

[illegible]

[illegible]

[illegible]

[illegible]

$$\begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \\ \theta_7 \\ \theta_8 \\ \theta_9 \\ \theta_{10} \\ \theta_{11} \\ \theta_{12} \\ \theta_{13} \\ \theta_{14} \\ \theta_{15} \\ \theta_{16} \\ \theta_{17} \\ \theta_{18} \\ \theta_{19} \\ \theta_{20} \\ \theta_{21} \\ \theta_{22} \\ \theta_{23} \\ \theta_{24} \\ \theta_{25} \\ \theta_{26} \\ \theta_{27} \\ \theta_{28} \\ \theta_{29} \\ \theta_{30} \\ \theta_{31} \\ \theta_{32} \\ \theta_{33} \\ \theta_{34} \\ \theta_{35} \\ \theta_{36} \\ \theta_{37} \\ \theta_{38} \\ \theta_{39} \\ \theta_{40} \\ \theta_{41} \\ \theta_{42} \\ \theta_{43} \\ \theta_{44} \\ \theta_{45} \\ \theta_{46} \\ \theta_{47} \\ \theta_{48} \\ \theta_{49} \\ \theta_{50} \\ \theta_{51} \\ \theta_{52} \\ \theta_{53} \\ \theta_{54} \\ \theta_{55} \\ \theta_{56} \\ \theta_{57} \\ \theta_{58} \\ \theta_{59} \\ \theta_{60} \\ \theta_{61} \\ \theta_{62} \\ \theta_{63} \\ \theta_{64} \\ \theta_{65} \\ \theta_{66} \\ \theta_{67} \\ \theta_{68} \\ \theta_{69} \\ \theta_{70} \\ \theta_{71} \\ \theta_{72} \\ \theta_{73} \\ \theta_{74} \\ \theta_{75} \\ \theta_{76} \\ \theta_{77} \\ \theta_{78} \\ \theta_{79} \\ \theta_{80} \\ \theta_{81} \\ \theta_{82} \\ \theta_{83} \\ \theta_{84} \\ \theta_{85} \\ \theta_{86} \\ \theta_{87} \\ \theta_{88} \\ \theta_{89} \\ \theta_{90} \\ \theta_{91} \\ \theta_{92} \\ \theta_{93} \\ \theta_{94} \\ \theta_{95} \\ \theta_{96} \\ \theta_{97} \\ \theta_{98} \\ \theta_{99} \end{bmatrix}$$

[illegible]

[illegible]

```

6,
6,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
9,
9,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
7,
7,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
6,
6,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
]

```


$$\begin{bmatrix} 0, \\ 0, \\ 0, \\ 0, \\ 0, \end{bmatrix},$$


```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
```


[illegible]

[illegible]


```
2,  
0,  
0,  
0,  
0,  
0,  
0,  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
2,  
2,  
0,  
0,  
0,  
0,  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
2,  
0,  
0,  
0,  
0,  
],  
[  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
0,  
],  
[  
0,  
0,  
0,  
0,  
0,
```


[illegible]


```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
2,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
2,
0,
0,
0,
0,
],
[
```



```

0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
7,
7,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
],
[
0,
0,
6,
6,
6,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0
]
],
"InputSHA256": "ae09606fc84d305c548c0ed40acbf977f4924187c253bc6a91767febd40b5c",
"QueryKind": "GLOBAL_BIT1",
"QueryLevel": null,
"QueryPrefix": [],
"AdmissionMode": "CONCEPT_GUIDED_EXACT_DP_WITH_FALLBACK",
"CertifiedMinimumDepthBefore": 1,
"BaselineCertifiedMinimumDepthBefore": 1,
"BaselineFirstInputSHA256": "ae09606fc84d305c548c0ed40acbf977f4924187c253bc6a91767febd40b5c",
"RootConceptRawMatched": true,
"RootConceptActivationEnvelopeRejected": true,
"RootConceptMatched": false,
"RootConceptIDs": [],
"RootConceptPreferredInputSHA256": [],
"SelectedQueryWasRootConceptPreference": false,
"RootConceptFallbackUsed": true,
"GuidedWorkCounters": {
  "ExpandedStateCount": 1,
  "QueryEvaluationCount": 17,
  "OutcomeBranchEvaluationCount": 49,
  "ConceptApplicableStateCount": 0,
  "ConceptInstantiatedQueryCount": 0,
  "ConceptNoCandidateStateCount": 1,
  "ConceptPreferredQueryRejectedCount": 0,
  "ConceptActivationEnvelopeRejectedStateCount": 1,
  "ConceptActivationEnvelopeRejectedQueryCount": 22
},

```


[illegible]

[illegible]

```
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
```

```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
2,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
0,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
2,
0,
0,
0,
0,
```


0,
0,
0,
0,
0,
0,
5[illegible][illegible][illegible]
$$\begin{bmatrix} 0, \\ 0, \\ 0, \\ 0, \end{bmatrix}$$

```
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
]
```



```
0,
0,
0,
2,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
2,
0,
0,
0,
0
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
2,
0,
0,
0,
0,
0
],
[
0,
0,
0,
```



```

    0,
    0
  ],
  [
    0,
    0,
    6,
    6,
    6,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0,
    0
  ]
],
"Status": "DECISION_CERTIFIED",
"TestOutputAccessed": false,
"HiddenProgramAccessedByLearner": false
},
{
  "TaskID": "KT002",
  "RuntimeSeconds": 1.9921863,
  "InitialSemanticClassCount": 12,
  "InitialDecisionClassCount": 3,
  "InitialPlanExistsWithinCap": true,
  "InitialCertifiedMinimumDepth": 2,
  "InitialBaselineCertifiedMinimumDepth": 2,
  "PairedDepthParity": true,
  "InitialGuidedWorkCounters": {
    "ExpandedStateCount": 3,
    "QueryEvaluationCount": 45,
    "OutcomeBranchEvaluationCount": 55,
    "ConceptApplicableStateCount": 3,
    "ConceptInstantiatedQueryCount": 8,
    "ConceptNoCandidateStateCount": 0,
    "ConceptPreferredQueryRejectedCount": 3,
    "ConceptActivationEnvelopeRejectedStateCount": 0,
    "ConceptActivationEnvelopeRejectedQueryCount": 0
  },
  "InitialBaselineWorkCounters": {
    "ExpandedStateCount": 8,
    "QueryEvaluationCount": 270,
    "OutcomeBranchEvaluationCount": 310,
    "ConceptApplicableStateCount": 0,
    "ConceptInstantiatedQueryCount": 0,
    "ConceptNoCandidateStateCount": 0,
    "ConceptPreferredQueryRejectedCount": 0,
    "ConceptActivationEnvelopeRejectedStateCount": 0,
    "ConceptActivationEnvelopeRejectedQueryCount": 0
  },
  "GuidedQueryEvaluationCount": 45,
  "BaselineQueryEvaluationCount": 270,
  "GuidedExpandedStateCount": 3,
  "BaselineExpandedStateCount": 8,
  "FinalSemanticClassCount": 2,
  "FinalDecisionClassCount": 1,
  "ActiveQueryCount": 1,
  "ActiveQueryTrace": [
    {
      "QueryNumber": "KQ01",
      "Input": [
        0,
        0,

```


[illegible]

$$\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_{n-1} \\ \theta_n \end{bmatrix}, \quad \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_{n-1} \\ \theta_n \end{bmatrix}$$

[illegible]

```
[
  [
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0],
    [0]
  ],
  {
    "InputSHA256": "0b66088c679d0d7c25e7e8d4002d390da95769f327a2effab55a0f80c9bc894a",
    "QueryKind": "DECISION",
    "QueryLevel": null,
    "QueryPrefix": [
      [1],
      [0],
      [1],
      [0]
    ],
    "AdmissionMode": "CONCEPT_GUIDED_EXACT_DP_WITH_FALLBACK",
    "CertifiedMinimumDepthBefore": 2,
    "BaselineCertifiedMinimumDepthBefore": 2,
    "BaselineFirstInputSHA256": "0b66088c679d0d7c25e7e8d4002d390da95769f327a2effab55a0f80c9bc894a",
    "RootConceptRawMatched": true,
    "RootConceptActivationEnvelopeRejected": false,
    "RootConceptMatched": true,
    "RootConceptIDs": [
      "KC_f5985bcf1108178",
      "KC_ed4b21f1f97412bc"
    ],
    "RootConceptPreferredInputSHA256": [
      "0b66088c679d0d7c25e7e8d4002d390da95769f327a2effab55a0f80c9bc894a",
      "2de4cfd428856e353fca8b1e61307dfe71f2167da83e409ab567d14fb6808744",
      "fe47143d75c80eb733ea97c820ef68f2068609cd95ae4be1ee61ebf31888151f"
    ],
    "SelectedQueryWasRootConceptPreference": true,
    "RootConceptFallbackUsed": false,
    "GuidedWorkCounters": {
      "ExpandedStateCount": 3,
      "QueryEvaluationCount": 45,
      "OutcomeBranchEvaluationCount": 55,
      "ConceptApplicableStateCount": 3,
      "ConceptInstantiatedQueryCount": 8,

```

[illegible]

$$\begin{bmatrix} \theta_0 \\ \theta_8 \\ \theta_9 \\ \theta_{10} \\ \theta_{11} \\ \theta_{12} \\ \theta_{13} \\ \theta_{14} \\ \theta_{15} \\ \theta_{16} \\ \theta_{17} \\ \theta_{18} \\ \theta_{19} \\ \theta_{20} \\ \theta_{21} \\ \theta_{22} \\ \theta_{23} \\ \theta_{24} \\ \theta_{25} \\ \theta_{26} \\ \theta_{27} \\ \theta_{28} \\ \theta_{29} \\ \theta_{30} \\ \theta_{31} \\ \theta_{32} \\ \theta_{33} \\ \theta_{34} \\ \theta_{35} \\ \theta_{36} \\ \theta_{37} \\ \theta_{38} \\ \theta_{39} \\ \theta_{40} \\ \theta_{41} \\ \theta_{42} \\ \theta_{43} \\ \theta_{44} \\ \theta_{45} \\ \theta_{46} \\ \theta_{47} \\ \theta_{48} \\ \theta_{49} \\ \theta_{50} \\ \theta_{51} \\ \theta_{52} \\ \theta_{53} \\ \theta_{54} \\ \theta_{55} \\ \theta_{56} \\ \theta_{57} \\ \theta_{58} \\ \theta_{59} \\ \theta_{60} \\ \theta_{61} \\ \theta_{62} \\ \theta_{63} \\ \theta_{64} \\ \theta_{65} \\ \theta_{66} \\ \theta_{67} \\ \theta_{68} \\ \theta_{69} \\ \theta_{70} \\ \theta_{71} \\ \theta_{72} \\ \theta_{73} \\ \theta_{74} \\ \theta_{75} \\ \theta_{76} \\ \theta_{77} \\ \theta_{78} \\ \theta_{79} \\ \theta_{80} \\ \theta_{81} \\ \theta_{82} \\ \theta_{83} \\ \theta_{84} \\ \theta_{85} \\ \theta_{86} \\ \theta_{87} \\ \theta_{88} \\ \theta_{89} \\ \theta_{90} \\ \theta_{91} \\ \theta_{92} \\ \theta_{93} \\ \theta_{94} \\ \theta_{95} \\ \theta_{96} \\ \theta_{97} \\ \theta_{98} \\ \theta_{99} \end{bmatrix},$$

$$\begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_{n-1} \end{bmatrix}, \quad \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_{n-1} \end{bmatrix}$$

[illegible]

```
0,
0,
0,
0,
0,
0,
0,
0,
1,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
1,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
1,
0,
0,
0,
],
[
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
0,
]
```


[illegible]

[illegible]

```

    ],
    "DecisionClassCountAfter": 1,
    "SemanticClassCountAfter": 2,
    "GeneratedByTCCKernel": true,
    "TestOutputAccessed": false,
    "HiddenProgramAccessedByLearner": false
  },
  "AdaptiveStopReason": "DECISION_CERTIFIED",
  "DecisionCertified": true,
  "TestPredictionCommitted": true,
  "CommittedTestPrediction": [

```

[illegible]

[illegible]

☐ Yes
☐ No

☐ Yes
☐ No

 $\theta,$

☐ Yes, I am

$$\left[\begin{array}{c}] \\ [\end{array} \right.$$

① ② ③ ④ ⑤ ⑥ ⑦ ⑧ ⑨ ⑩ ⑪ ⑫ ⑬ ⑭ ⑮ ⑯ ⑰ ⑱ ⑲ ⑳ ㉑ ㉒ ㉓ ㉔ ㉕ ㉖ ㉗ ㉘ ㉙ ㉚ ㉛ ㉜ ㉝ ㉞ ㉟ ㊱ ㊲ ㊳ ㊴ ㊵ ㊶ ㊷ ㊸ ㊹ ㊺ ㊻ ㊼ ㊽ ㊾ ㊿

$$\left[\begin{array}{c}] \\ [\end{array} \right],$$

①, ②, ③, ④, ⑤, ⑥, ⑦, ⑧, ⑨, ⑩, ⑪, ⑫, ⑬, ⑭, ⑮, ⑯, ⑰, ⑱, ⑲, ⑳, ㉑, ㉒, ㉓, ㉔, ㉕, ㉖, ㉗, ㉘, ㉙, ㉚, ㉛, ㉜, ㉝, ㉞, ㉟, ㊱, ㊲, ㊳, ㊴, ㊵, ㊶, ㊷, ㊸, ㊹, ㊺, ㊻, ㊼, ㊽, ㊾, ㊿

$$\left. \begin{array}{l}] \\ [\end{array} \right\}$$
$$\theta,$$

[illegible]

$$\begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n-1} \end{pmatrix}, \quad \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{n-1} \end{pmatrix}$$

$\Theta,$
 $\Theta,$
 $\Theta,$
 $\Theta,$
 $\Theta,$


```
"NativePreScorePass": true,  
"CoreRewriteFreezeDedupModified": false,  
"Conclusion": "KERNEL_NATIVE_CONCEPT_FORMATION_COMPLETE_AWAITING_SEALED_SCORE"  
}
```

Independent replay verification

```
{
  "Stage": "S140-C DIAGNOSTIC R1 source-selectivity activation envelope replay independent replay verification",
  "ProtocolSHA256": "6ef80df675aeda0f9b5c0333499a9abd1663cf9777331a54e2741ab12fec3b83",
  "OriginalS140BFailureStillValid": true,
  "RetrospectiveReplayOnly": true,
  "ActivationEnvelope": {
    "Rule": "PREFERRED_FRACTION_NOT_ABOVE_MAXIMUM_SOURCE_EVENT_FRACTION",
    "MaximumSourcePreferredFractionNumerator": 7,
    "MaximumSourcePreferredFractionDenominator": 30,
    "SourceEventFractions": [
      {
        "TaskID": "SRC001",
        "Numerator": 7,
        "Denominator": 30
      },
      {
        "TaskID": "SRC002",
        "Numerator": 3,
        "Denominator": 43
      },
      {
        "TaskID": "SRC002",
        "Numerator": 1,
        "Denominator": 42
      },
      {
        "TaskID": "SRC003",
        "Numerator": 1,
        "Denominator": 20
      }
    ]
  },
  "TargetComparisons": [
    {
      "TaskID": "KT005",
      "Role": "NO_REUSABLE_CONCEPT_FALLBACK",
      "Exact": true,
      "ExpectedDepth": 2,
      "ObservedDepth": 2,
      "GuidedWork": 11,
      "BaselineWork": 11,
      "OriginalFailedGuidedWork": 6,
      "EnvelopeRejectedStateCount": 4,
      "RootFallbackCount": 2
    },
    {
      "TaskID": "KT001",
      "Role": "SURFACE_PERMUTATION_TRANSFER",
      "Exact": true,
      "ExpectedDepth": 1,
      "ObservedDepth": 1,
      "GuidedWork": 1,
      "BaselineWork": 14,
      "OriginalFailedGuidedWork": 1,
      "EnvelopeRejectedStateCount": 0,
      "RootFallbackCount": 0
    },
    {
      "TaskID": "KT003",
      "Role": "SEQUENTIAL_CONCEPT_COMPOSITION",
      "Exact": true,
      "ExpectedDepth": 3,
      "ObservedDepth": 3,
      "GuidedWork": 1757,
      "BaselineWork": 4198,
      "OriginalFailedGuidedWork": 9344,
      "EnvelopeRejectedStateCount": 32,
      "RootFallbackCount": 1
    }
  ]
}
```

```

    },
    {
      "TaskID": "KT004",
      "Role": "ADVERSARIAL_CONCEPT_REJECTION",
      "Exact": true,
      "ExpectedDepth": 2,
      "ObservedDepth": 2,
      "GuidedWork": 235,
      "BaselineWork": 235,
      "OriginalFailedGuidedWork": 626,
      "EnvelopeRejectedStateCount": 10,
      "RootFallbackCount": 2
    },
    {
      "TaskID": "KT002",
      "Role": "COORDINATE_SCALE_TRANSFER",
      "Exact": true,
      "ExpectedDepth": 2,
      "ObservedDepth": 2,
      "GuidedWork": 45,
      "BaselineWork": 270,
      "OriginalFailedGuidedWork": 45,
      "EnvelopeRejectedStateCount": 0,
      "RootFallbackCount": 0
    }
  ],
  "Checks": {
    "RetrospectiveStatusExplicit": true,
    "OriginalFailureHashesPreserved": true,
    "TaskBodiesUnchanged": true,
    "FrozenRepairHashesPass": true,
    "NativePythonConceptBodiesMatch": true,
    "SourceDerivedActivationEnvelopeMatch": true,
    "AllFiveAnswersAndDepthsExact": true,
    "TransferWorkBelowBaseline": true,
    "SequentialFailureRepaired": true,
    "NoReuseControlFallsBack": true,
    "EnvelopeActuallyExercised": true,
    "NativePreScorePass": true,
    "CoreUnchanged": true
  },
  "RepairReplayPass": true,
  "CoreRewriteFreezeDedupModified": false,
  "Conclusion": "RETROSPECTIVE_HIGH_PREFIX_ROUTER_REPAIR_PASS"
}

```